

Disjoint Set Union (DSU) / Union-Find

1. Overview

Disjoint Set Union (DSU), also called Union-Find, is a data structure that tracks a set of elements partitioned into **disjoint** (non-overlapping) subsets. It supports two main operations:

- **Find**: Determine which subset a particular element belongs to.
- **Union**: Merge two subsets into a single subset.

Think of it as managing friend groups at a party. Initially, everyone is their own group. When two people become friends, their groups merge. You can ask "are these two people in the same friend group?" at any time.

DSU is one of the most elegant and widely used data structures in competitive programming and graph problems.

2. Intuition

Simple Explanation

Imagine you have n independent nodes. Each node initially points to itself (its own parent). When you connect two nodes, you make one point to the other (or to a common root). The **root** of a node is the representative of its set. Two nodes are in the same set if they share the same root.

Analogy: Family Tree / Clan System

- Each person starts as the head of their own clan.
- When two clans merge, the smaller clan's head pledges allegiance to the larger clan's head (union by rank/size).
- To find who the ultimate leader of a person is, you follow the chain of allegiance upward (find operation).
- Along the way, you flatten the chain so next time it's faster (path compression).

Step-by-Step Reasoning

1. Each element is a node in a forest of trees.
2. Each tree represents one set. The root of the tree is the "representative" of that set.
3. `find(x)` climbs up the parent pointers until it reaches the root.
4. `union(x, y)` finds roots of both, then attaches one root under the other.
5. **Path compression** flattens the tree during `find`, making future finds $O(\alpha(n))$.
6. **Union by rank/size** keeps trees shallow by attaching smaller tree under larger tree.

Why It Works

DSU is efficient because of two optimizations:

- **Path compression** ensures that after a few operations, most nodes point directly to the root.

- **Union by rank/size** prevents tall trees from forming.

Together, these make the amortized time per operation **inverse Ackermann** — effectively constant for all practical input sizes.

3. When to Use It

Use DSU when you need to:

- **Track connected components** in a dynamic graph (edges added over time).
- **Check if two nodes are in the same set** efficiently.
- **Merge groups** repeatedly.
- **Detect cycles** in an undirected graph.
- **Find number of connected components** after a series of union operations.
- **Solve problems involving equivalence relations** (e.g., "friends" relation, "connected" relation).

Common Trigger Phrases in Problems

- "Find if two elements are in the same group"
- "Merge groups/sets"
- "Number of connected components"
- "Minimum spanning tree" (Kruskal's algorithm)
- "Dynamic connectivity"
- "Friends/foes" or "enemy of my enemy"
- "Equations with variables" (satisfiability)
- "Redundant connection"
- "Accounts merge"

4. When Not to Use It

Situation	Why DSU is not suitable	Alternative
Need to split/separate sets	DSU only supports union, not split.	Link-cut tree, or offline reverse processing
Need to query the actual elements in a set	DSU only tells you the representative, not the set contents.	Maintain a list for each set (union by size + merge lists)
Need range queries or path queries on a tree	DSU is for connectivity, not path queries.	Segment tree, Fenwick tree, LCA
Dynamic graph with deletions	DSU cannot handle edge deletions natively.	DSU with rollback (offline), link-cut tree
Graph is dense and all edges are known upfront	BFS/DFS is simpler and as fast.	BFS / DFS
Need shortest path	DSU does not handle distances.	Dijkstra, Floyd-Warshall, BFS

Situation	Why DSU is not suitable	Alternative
Small, static graph	Overkill. A simple DFS or adjacency matrix is enough.	DFS / adjacency matrix

Common Wrong Assumptions

- DSU does **not** maintain the order of elements.
- DSU does **not** give you the path between two nodes, only whether they are connected.
- DSU without path compression (naive) is $O(n)$ per operation — slow for large inputs.

5. Core Concepts

5.1 Parent Array

`parent[i]` stores the parent of node `i`. Initially, `parent[i] = i` (each node is its own parent).

A node is a **root** (representative) if `parent[i] == i`.

5.2 Find Operation

Returns the root of the set containing `x`.

```
int find(int x) {
    if (parent[x] == x) return x;
    return find(parent[x]);    // naive
}
```

5.3 Union Operation

Merges the sets containing `x` and `y`.

```
void unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx != ry) parent[ry] = rx;    // naive
}
```

5.4 Path Compression

During `find`, make every node on the path point directly to the root.

```
int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}
```

5.5 Union by Rank

Keep track of tree height (rank). Attach shorter tree under taller tree.

```
if (rank[rx] < rank[ry]) swap(rx, ry);
parent[ry] = rx;
if (rank[rx] == rank[ry]) rank[rx]++;
```

5.6 Union by Size

Keep track of size of each set. Attach smaller set under larger set.

```
if (size[rx] < size[ry]) swap(rx, ry);
parent[ry] = rx;
size[rx] += size[ry];
```

5.7 Connected Components

Initially, `components` = `n`. Each successful union decreases `components` by 1.

5.8 Cycle Detection

In an undirected graph, if `find(u) == find(v)` before uniting, then adding edge `(u, v)` creates a cycle.

6. Step-by-Step Algorithm

Standard DSU (with path compression + union by size)

Initialize:

1. Create `parent` array of size `n` where `parent[i] = i`.
2. Create `size` array of size `n` where `size[i] = 1`.
3. Set `components = n`.

Find(x):

1. If `parent[x] == x`, return `x`.
2. Else, recursively find parent of `parent[x]` and set `parent[x] = find(parent[x])`.
3. Return `parent[x]`.

Union(x, y):

1. Find roots: `rx = find(x)`, `ry = find(y)`.
2. If `rx == ry`, do nothing (already in same set).
3. If `size[rx] < size[ry]`, swap `rx` and `ry` so `rx` is the larger root.
4. Set `parent[ry] = rx`.
5. Update `size[rx] += size[ry]`.

6. Decrement **components** by 1.

Connected(x, y):

1. Return **find(x) == find(y)**.

7. Dry Run

Problem: Merge groups and check connectivity

Initial: **n = 7** (nodes 0..6)

Operations:

1. Union(0, 1)
2. Union(2, 3)
3. Union(1, 2)
4. Connected(0, 3) → ?
5. Connected(0, 4) → ?

Initial State

Node	0	1	2	3	4	5	6
Parent	0	1	2	3	4	5	6
Size	1	1	1	1	1	1	1

Components: 7

Step 1: Union(0, 1)

- **find(0) = 0, find(1) = 1**
- Sizes: size[0]=1, size[1]=1 → no swap.
- **parent[1] = 0, size[0] = 2**

Node	0	1	2	3	4	5	6
Parent	0	0	2	3	4	5	6
Size	2	1	1	1	1	1	1

Components: 6

Step 2: Union(2, 3)

- **find(2) = 2, find(3) = 3**
- Sizes: size[2]=1, size[3]=1 → no swap.
- **parent[3] = 2, size[2] = 2**

Node	0	1	2	3	4	5	6
Parent	0	0	2	2	4	5	6
Size	2	1	2	1	1	1	1

Components: 5

Step 3: Union(1, 2)

- `find(1)`: `parent[1] = 0`, `parent[0] = 0` → `root = 0`
- `find(2)`: `parent[2] = 2` → `root = 2`
- Sizes: `size[0]=2`, `size[2]=2` → no swap (equal).
- `parent[2] = 0`, `size[0] = 4`

Node	0	1	2	3	4	5	6
Parent	0	0	0	2	4	5	6
Size	4	1	2	1	1	1	1

Components: 4

Step 4: Connected(0, 3)

- `find(0) = 0`
- `find(3)`: `parent[3] = 2`, `parent[2] = 0`, `parent[0] = 0` → `root = 0` (path compression sets `parent[3]=0`)
- `find(0) == find(3)` → **True** ✓

Step 5: Connected(0, 4)

- `find(0) = 0`
- `find(4) = 4`
- `0 != 4` → **False** ✗

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class DSU {
private:
    vector<int> parent, sz;    // parent array and size array

public:
    // Constructor: initialize n elements
    DSU(int n) {
        parent.resize(n);
        sz.resize(n, 1);
        for (int i = 0; i < n; i++) parent[i] = i;
    }
};
```

```

    }

    // Find with path compression
    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }

    // Union by size
    void unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return;

        // Attach smaller tree under larger tree
        if (sz[rx] < sz[ry]) swap(rx, ry);
        parent[ry] = rx;
        sz[rx] += sz[ry];
    }

    // Check if two elements are in the same set
    bool connected(int x, int y) {
        return find(x) == find(y);
    }

    // Get size of the set containing x
    int getSize(int x) {
        return sz[find(x)];
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n = 7;
    DSU dsu(n);

    dsu.unite(0, 1);
    dsu.unite(2, 3);
    dsu.unite(1, 2);

    cout << "0 and 3 connected? " << (dsu.connected(0, 3) ? "Yes" : "No") << "\n";
    // Yes
    cout << "0 and 4 connected? " << (dsu.connected(0, 4) ? "Yes" : "No") << "\n";
    // No
    cout << "Size of set containing 0: " << dsu.getSize(0) << "\n";
    // 4

    return 0;
}

```

9. Python Implementation

```

class DSU:
    def __init__(self, n: int):
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        # Path compression
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x
        # Recursive version:
        # if self.parent[x] != x:
        #     self.parent[x] = self.find(self.parent[x])
        # return self.parent[x]

    def unite(self, x: int, y: int) -> bool:
        # Returns True if a merge happened, False if already in same set
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False

        # Union by size
        if self.size[rx] < self.size[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        self.size[rx] += self.size[ry]
        return True

    def connected(self, x: int, y: int) -> bool:
        return self.find(x) == self.find(y)

    def get_size(self, x: int) -> int:
        return self.size[self.find(x)]

# Example usage
if __name__ == "__main__":
    dsu = DSU(7)
    dsu.unite(0, 1)
    dsu.unite(2, 3)
    dsu.unite(1, 2)

    print("0 and 3 connected?", dsu.connected(0, 3)) # True
    print("0 and 4 connected?", dsu.connected(0, 4)) # False
    print("Size of set containing 0:", dsu.get_size(0)) # 4

```

10. Code Explanation

Class Structure

```
class DSU {  
private:  
    vector<int> parent, sz;
```

- `parent`: stores the parent of each node. A node is a root if `parent[i] == i`.
- `sz`: stores the size of the set for root nodes. Only meaningful for roots.

Constructor

```
DSU(int n) {  
    parent.resize(n);  
    sz.resize(n, 1);  
    for (int i = 0; i < n; i++) parent[i] = i;  
}
```

- Each node is its own parent initially.
- Each set has size 1.
- Time: $O(n)$

Find with Path Compression

```
int find(int x) {  
    if (parent[x] != x)  
        parent[x] = find(parent[x]);  
    return parent[x];  
}
```

- Recursively follows parent pointers until root.
- **Path compression**: on the way back, every visited node is attached directly to the root.
- This is what makes DSU nearly $O(1)$ amortized.

Union by Size

```
void unite(int x, int y) {  
    int rx = find(x), ry = find(y);  
    if (rx == ry) return;  
  
    if (sz[rx] < sz[ry]) swap(rx, ry);  
    parent[ry] = rx;  
    sz[rx] += sz[ry];  
}
```

- First find roots of both elements.
- If same root, they are already connected → return.
- Attach the smaller tree under the larger tree.
- Update size of the new root.

Connected Check

```
bool connected(int x, int y) {
    return find(x) == find(y);
}
```

- Simply checks if both have the same root.

Get Size

```
int getSize(int x) {
    return sz[find(x)];
}
```

- Returns the size of the set containing x .

11. Complexity Analysis

Operation	Without Optimization	With Path Compression Only	With Path Compression + Union by Rank/Size
<code>find</code>	$O(n)$	$O(\log n)$ amortized	$O(\alpha(n))$ amortized
<code>unite</code>	$O(n)$	$O(\log n)$ amortized	$O(\alpha(n))$ amortized
<code>connected</code>	$O(n)$	$O(\log n)$ amortized	$O(\alpha(n))$ amortized
Construction	$O(n)$	$O(n)$	$O(n)$
Space	$O(n)$	$O(n)$	$O(n)$

$\alpha(n)$ is the inverse Ackermann function. For all practical values of n (up to 10^{600}), $\alpha(n) \leq 5$. This is essentially constant time.

Aspect	Complexity
Time per operation (amortized)	$O(\alpha(n)) \approx O(1)$
Space	$O(n)$
Worst-case single find (without path compression)	$O(n)$
Worst-case single find (with both optimizations)	$O(\log n)$

12. Common Patterns

Pattern 1: Connected Components Count

How to identify: Problem asks for number of groups/components after a series of unions.

General approach: Start with `components = n`. Decrement by 1 on each successful union.

Example problems: Number of Provinces (LeetCode), Accounts Merge (LeetCode)

Pattern 2: Cycle Detection in Undirected Graph

How to identify: Adding edges one by one, check if adding an edge creates a cycle.

General approach: Before union, check if `find(u) == find(v)`. If yes, adding this edge creates a cycle.

Example problems: Redundant Connection (LeetCode), Graph Valid Tree (LeetCode)

Pattern 3: Minimum Spanning Tree (Kruskal's Algorithm)

How to identify: Need minimum cost to connect all nodes.

General approach: Sort edges by weight. Process edges in increasing order. If edge connects two different components, add it to MST.

Example problems: Kruskal's MST (CSES), Connecting Cities (LeetCode)

Pattern 4: Offline Queries with DSU

How to identify: Queries about connectivity after certain edges are removed, or queries at different time points.

General approach: Process queries in reverse order (add edges instead of removing them).

Example problems: Checking Existence of Edge Length Limited Paths (LeetCode), Number of Islands II (LeetCode)

Pattern 5: DSU on Grid

How to identify: 2D grid where cells are being activated or connected. Need to count islands/components.

General approach: Map each cell `(r, c)` to a unique node index `r * cols + c`. Use DSU on these indices.

Example problems: Number of Islands II (LeetCode), Bricks Falling When Hit (LeetCode)

Pattern 6: DSU with Rollback

How to identify: Need to undo unions, used in offline dynamic connectivity or divide-and-conquer on time intervals.

General approach: Store a stack of changes. On rollback, pop changes and revert parent/size arrays.

Example problems: Dynamic Connectivity (Codeforces), Offline Dynamic Connectivity (AtCoder)

Pattern 7: Persistent DSU

How to identify: Need to query connectivity at different historical versions.

General approach: Store parent array as a persistent data structure (array of versions). Each union creates a new version.

Example problems: Persistent Union Find (Codeforces), Historical Queries

13. Common Mistakes

1. Forgetting Path Compression

```
// BAD: no path compression
int find(int x) {
    while (parent[x] != x) x = parent[x];
    return x;
}
```

Without path compression, `find` can become $O(n)$ per call.

2. Not Calling `find` Inside `unite`

```
// BAD: using x and y directly instead of their roots
void unite(int x, int y) {
    if (parent[x] == parent[y]) return; // WRONG
    parent[y] = x;                      // WRONG
}
```

Always get the roots first.

3. Swapping Based on Uncompressed Parent

```
// BAD: comparing size of x and y, not their roots
if (size[x] < size[y]) swap(x, y);
```

Find roots first, then compare sizes of roots.

4. Wrong Initialization

- Forgetting to initialize `parent[i] = i` for all `i`.
- Forgetting to initialize `size[i] = 1` or `rank[i] = 0`.

5. Union by Rank Confusion

Using rank (tree height) vs size (number of nodes). Both work, but:

- Union by rank uses the estimated height of the tree.
- Union by size uses the number of elements.

6. Not Handling 0-Indexed vs 1-Indexed

Some problems use 1-indexed nodes. Adjust DSU size accordingly ($n+1$).

7. Recursion Depth for Find

Recursive `find` can cause stack overflow for very deep recursion (though path compression makes this rare). Use iterative version if worried.

8. Modifying Parent Without Path Compression

```
// BAD: modifies parent but doesn't compress
int find(int x) {
    if (parent[x] != x) return find(parent[x]); // no assignment
    return x;
}
```

9. Forgetting `const` or Reference in Parameters

Not a correctness issue, but good practice.

14. Edge Cases

Edge Case	What to Check
$n = 0$	No elements. DSU is empty.
$n = 1$	Single element. All operations trivially succeed.
No unions	Each element is its own set. <code>find(i) == i</code> for all.
All elements already connected	Every <code>unite</code> returns early (already same set).
Union with self	<code>unite(x, x)</code> should be a no-op.
Maximum n	Stack overflow for recursive find? Memory limit?
Disconnected graph	Multiple components remain.
Complete graph	All nodes connected.
Union order dependent	Some problems expect specific merging behavior.
1-indexed nodes	Pass $n+1$ to DSU constructor.

15. Variations

15.1 DSU with Rollback

What changes: Instead of permanently modifying parent and size, push old values onto a stack. `rollback()` restores the previous state.

When used: Offline dynamic connectivity problems, divide and conquer on time intervals.

Placement relevance: Medium. Rare in interviews, but common in CP.

15.2 Persistent DSU

What changes: Each union creates a new version. You can query connectivity at any historical version.

When used: Problems requiring queries at different time points.

Placement relevance: Low for placements, medium for CP.

15.3 DSU with Path Halving / Splitting

What changes: Alternative path compression strategies that modify pointers during traversal without recursion.

```
// Path halving (iterative)
int find(int x) {
    while (parent[x] != x) {
        parent[x] = parent[parent[x]];
        x = parent[x];
    }
    return x;
}
```

When used: When avoiding recursion is important.

15.4 DSU with Distance to Root

What changes: Maintain an additional array `dist[x]` that stores the distance from `x` to its parent. Useful for problems where you need relative distances within a set.

When used: Problems like "friend of a friend" or parity-based grouping.

Placement relevance: Medium. Appears in problems like "Possible Bipartition."

15.5 Reversible DSU

What changes: Both union and undo operations are supported. Requires storing the entire history of changes.

When used: When you need to support both union and rollback operations online.

15.6 DSU on Grid (2D to 1D mapping)

What changes: Map (r, c) to $r * cols + c$ and use standard DSU.

When used: Grid-based connectivity problems, islands, wall-busting.

16. Related Algorithms/Data Structures

Algorithm/DS	Connection	When to Choose
DFS/BFS	Also finds connected components	DSU is better when edges are added dynamically
Kruskal's MST	Uses DSU internally	DSU is a building block, not a replacement
Link-Cut Tree	Dynamic tree connectivity with splits	Use LCT when you need to cut edges; DSU is simpler for union-only
Segment Tree	Range queries	DSU is for connectivity, not for range queries
Bipartite Check (DFS)	2-coloring	Use DSU with parity for dynamic bipartite checks
Tarjan's SCC	Strongly connected components	DSU handles undirected; Tarjan for directed

DSU vs DFS for Connected Components

Aspect	DSU	DFS
Dynamic edges (added over time)	✔ Natural	✗ Need to restart
Static graph	$O(\alpha(n))$ per union	$O(n + m)$ once
Memory	$O(n)$	$O(n + m)$
Implementation	Simple	Simple
Best for	Online queries, dynamic	Static graph, all components at once

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
Number of Provinces	LeetCode 547	Count connected components in an adjacency matrix	Easy
Redundant Connection	LeetCode 684	Find edge that creates a cycle in a graph	Easy

Medium

Problem	Platform	Idea	Difficulty
Accounts Merge	LeetCode 721	Merge accounts that share email addresses using DSU on emails	Medium
Graph Valid Tree	LeetCode 261	Check if graph is a tree (connected + no cycles)	Medium
Number of Islands II	LeetCode 305	DSU on grid; add islands one by one and count components	Medium

Hard

Problem	Platform	Idea	Difficulty
Bricks Falling When Hit	LeetCode 803	Reverse processing + DSU on grid	Hard
Checking Existence of Edge Length Limited Paths	LeetCode 1697	Offline queries sorted by limit, DSU with union by weight	Hard
Dynamic Connectivity	Codeforces (CF 813E)	DSU with rollback, divide and conquer on time intervals	Hard

18. Interview Explanation

"Disjoint Set Union, or Union-Find, is a data structure that maintains a collection of disjoint sets. It supports two operations in nearly constant time — $O(\alpha(n))$, where α is the inverse Ackermann function.

The core idea is a forest of trees where each tree represents a set. Each node has a parent pointer, and the root of the tree is the representative of the set. Two optimizations make it fast: path compression, which flattens the tree during find operations, and union by rank or size, which keeps the tree shallow by attaching the smaller tree under the larger one.

I typically use it for problems involving dynamic connectivity — adding edges over time and checking if two nodes are in the same component. It's also the backbone of Kruskal's algorithm for Minimum Spanning Trees and cycle detection in undirected graphs.

The implementation is straightforward: a parent array, a size or rank array, a find function with path compression, and a union function that merges two sets. In practice, I can write this from memory in under a minute."

19. Revision Notes

- **Key idea:** Maintain disjoint sets as trees. Root = representative.
- **Operations:** `find(x)` → root of x; `unite(x, y)` → merge sets.
- **Path compression:** During find, set `parent[x] = find(parent[x])`.
- **Union by size:** Attach smaller set under larger set.
- **Complexity:** $O(\alpha(n))$ per operation $\approx O(1)$.
- **Cycle detection:** `find(u) == find(v)` before union → cycle exists.
- **Connected components count:** Start with `n`, decrement on each successful union.

- **DSU on grid:** Map $(r, c) \rightarrow r * \text{cols} + c$.
- **Common trap:** Always find roots first in `unite`. Never use `x` or `y` directly.
- **Memory:** $O(n)$ — two arrays of size n .
- **Template size:** ~15 lines of code.

20. Final Cheat Sheet

DSU / UNION-FIND CHEAT SHEET

WHEN TO USE

- Dynamic connectivity (edges added over time)
- Cycle detection in undirected graphs
- Kruskal's MST
- Connected components count
- Grid connectivity problems

MAIN OPERATIONS

`find(x)` → root of x's set $O(\alpha(n))$
`unite(x,y)` → merge sets of x and y $O(\alpha(n))$
`connected(x,y)` → are x and y in same set? $O(\alpha(n))$

COMPLEXITY

Time per operation: $O(\alpha(n)) \approx O(1)$
 Space: $O(n)$

KEY CODE (C++)

```

int find(int x) {
    return parent[x] == x ? x : (parent[x] = find(parent[x]));
}

void unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx == ry) return;
    if (size[rx] < size[ry]) swap(rx, ry);
    parent[ry] = rx;
    size[rx] += size[ry];
}

```

EDGE CASES

- $n = 0$, $n = 1$
- Self-union: `unite(x, x)`
- 1-indexed: allocate $n+1$
- Already connected: union returns early

COMMON TRAPS

- Forgetting path compression → $O(n)$ find
- Not finding roots before union
- Comparing size of x, y instead of roots
- 0-index vs 1-index confusion

MUST-KNOW PATTERNS

- DSU on grid: $(r,c) \rightarrow r \cdot \text{cols} + c$
- Reverse processing for deletions
- Offline queries sorted by threshold
- Kruskal's: sort edges, use DSU

Union by Rank / Size

1. Overview

Union by Rank and Union by Size are optimization strategies for the Union operation in DSU. They ensure that when merging two sets, the smaller tree is attached under the larger tree, keeping trees shallow.

Union by Size attaches the tree with fewer nodes under the tree with more nodes. **Union by Rank** attaches the tree with smaller height (rank) under the tree with larger height.

Both achieve the same asymptotic complexity.

2. Intuition

Simple Explanation

Imagine you're merging two groups of people. If you make the leader of the smaller group report to the leader of the larger group, the chain of command stays short. If you did the opposite, the chain could grow long, making it slow to find the ultimate leader.

Why It Works

Without this optimization, union is arbitrary — we might attach a large tree under a small one, creating a tall tree. A tall tree means **find** takes longer ($O(n)$ in worst case). By always attaching the smaller tree under the larger one, we ensure that the depth of any node is at most $O(\log n)$.

Proof sketch: When a node's depth increases by 1, its set size at least doubles. So depth can increase at most $O(\log n)$ times.

3. When to Use It

- **Always** use union by rank or size with DSU.
 - Without it, DSU is slow and unreliable.
 - Use it in every DSU implementation by default.
-

4. When Not to Use It

- When you need explicit control over which root becomes the new root (for some problem-specific reason).
- In that case, you can skip it, but understand that find operations will be slower.

5. Core Concepts

5.1 Size Array

`size[i]` stores the number of elements in the set whose root is `i`. Only meaningful for root nodes.

5.2 Rank Array

`rank[i]` stores an upper bound on the height of the tree rooted at `i`. Only meaningful for root nodes.

5.3 Size vs Rank

Aspect	Union by Size	Union by Rank
Tracks	Number of elements	Tree height (approx)
Update	<code>size[root] += size[child]</code>	<code>rank[root]++</code> only if equal
Pros	Gives you set size for free	Slightly smaller theoretical depth
Cons	Need extra array	Rank is approximate

Both give $O(\alpha(n))$ amortized time.

6. Step-by-Step Algorithm

Union by Size:

1. Find roots `rx` and `ry` of `x` and `y`.
2. If `rx == ry`, return.
3. If `size[rx] < size[ry]`, swap so `rx` is the larger root.
4. Set `parent[ry] = rx`.
5. Set `size[rx] += size[ry]`.

Union by Rank:

1. Find roots `rx` and `ry` of `x` and `y`.
2. If `rx == ry`, return.
3. If `rank[rx] < rank[ry]`, swap so `rx` has larger or equal rank.
4. Set `parent[ry] = rx`.
5. If `rank[rx] == rank[ry]`, increment `rank[rx]` by 1.

7. Dry Run

Union by Size on $n=5$

Initial: `parent = [0,1,2,3,4]`, `size = [1,1,1,1,1]`

Union(0, 1): sizes $1=1 \rightarrow$ no swap. `parent[1]=0`, `size[0]=2` **Union(2, 3):** sizes $1=1 \rightarrow$ no swap.

`parent[3]=2`, `size[2]=2` **Union(0, 2):** sizes: `size[0]=2`, `size[2]=2` $\rightarrow$ no swap. `parent[2]=0`, `size[0]=4`

Union(0, 4): sizes: `size[0]=4`, `size[4]=1` $\rightarrow$ no swap. `parent[4]=0`, `size[0]=5`

Final tree depth: at most 2 (root 0 $\rightarrow$ child 2 $\rightarrow$ child 3 is one path).

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class DSUBySize {
    vector<int> parent, sz;
public:
    DSUBySize(int n) {
        parent.resize(n);
        sz.resize(n, 1);
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]]; // path halving
            x = parent[x];
        }
        return x;
    }

    void unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return;
        if (sz[rx] < sz[ry]) swap(rx, ry);
        parent[ry] = rx;
        sz[rx] += sz[ry];
    }

    int getSize(int x) { return sz[find(x)]; }
};
```

9. Python Implementation

```
class DSUBySize:
    def __init__(self, n: int):
        self.parent = list(range(n))
        self.size = [1] * n
```

```

def find(self, x: int) -> int:
    # Iterative path compression
    while self.parent[x] != x:
        self.parent[x] = self.parent[self.parent[x]]
        x = self.parent[x]
    return x

def unite(self, x: int, y: int) -> bool:
    rx, ry = self.find(x), self.find(y)
    if rx == ry:
        return False
    if self.size[rx] < self.size[ry]:
        rx, ry = ry, rx
    self.parent[ry] = rx
    self.size[rx] += self.size[ry]
    return True

def get_size(self, x: int) -> int:
    return self.size[self.find(x)]

```

10. Complexity Analysis

Variant	find	unite	Space
No optimization	$O(n)$	$O(n)$	$O(n)$
Union by rank/size only	$O(\log n)$	$O(\log n)$	$O(n)$
Path compression + union by rank/size	$O(\alpha(n))$	$O(\alpha(n))$	$O(n)$

11. Common Mistakes

- Comparing size of x and y instead of their roots.
- Forgetting to update size/rank after union.
- Using union by rank but not incrementing rank when equal.
- Using union by size but not updating $size$ of the new root.

12. Edge Cases

- Both sets have equal size/rank → attach either way, increment rank if using rank.
- Single element sets → trivial.
- Already connected sets → early return.

Path Compression

1. Overview

Path Compression is an optimization for the **find** operation in DSU. After finding the root of a node, it makes every node on the path point directly to the root, flattening the tree.

2. Intuition

Simple Explanation

When you ask "who is the leader of this group?" and follow a chain of people, you remember everyone you passed. Next time someone asks about any of those people, you can answer immediately because you've told each person to remember the leader directly.

Analogy: Remembering the Shortcut

You walk through a maze to find the exit. Instead of remembering the entire path, you mark every room you pass with a direct arrow to the exit. Next time you're in any of those rooms, you see the exit immediately.

3. When to Use It

- **Always** use path compression with DSU.
 - Without it, DSU degrades to $O(\log n)$ or $O(n)$ per operation.
 - It's the single most important optimization for DSU.
-

4. Core Concepts

4.1 Recursive Path Compression

```
int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}
```

4.2 Iterative Path Compression (Path Halving)

```
int find(int x) {
    while (parent[x] != x) {
        parent[x] = parent[parent[x]]; // skip one level
        x = parent[x];
    }
    return x;
}
```

4.3 Iterative Path Compression (Full)

```

int find(int x) {
    int root = x;
    while (parent[root] != root) root = parent[root];
    while (x != root) {
        int next = parent[x];
        parent[x] = root;
        x = next;
    }
    return root;
}

```

5. Step-by-Step

find(5) with parent chain: $5 \rightarrow 3 \rightarrow 1 \rightarrow 0$

1. $\text{parent}[5] = 3 \neq 5 \rightarrow$ go deeper
2. $\text{parent}[3] = 1 \neq 3 \rightarrow$ go deeper
3. $\text{parent}[1] = 0 \neq 1 \rightarrow$ go deeper
4. $\text{parent}[0] = 0 \rightarrow$ root = 0
5. Backtrack: set $\text{parent}[1] = 0, \text{parent}[3] = 0, \text{parent}[5] = 0$
6. Return 0

Now $\text{parent}[5] = 0, \text{parent}[3] = 0, \text{parent}[1] = 0$. All point directly to root.

6. Complexity Impact

Without Path Compression	With Path Compression
$\text{find} = O(\log n)$ worst	$\text{find} = O(\alpha(n))$ amortized
Tree can be tall	Tree is always nearly flat
Each find traverses entire chain	After first find, chain is compressed

7. Common Mistakes

- Recursive find can overflow the stack for very deep recursion (mitigated by path compression itself).
- Forgetting to assign $\text{parent}[x] = \text{find}(\text{parent}[x])$ — just calling **find** recursively without saving the result doesn't compress.

Connected Components (with DSU)

1. Overview

A connected component of an undirected graph is a maximal set of vertices where every pair is connected by a path. DSU is an efficient way to track connected components as edges are added dynamically.

2. Intuition

Every time you union two nodes, they become part of the same component. By keeping a counter of the number of roots (or the number of successful unions), you can track how many components exist at any point.

3. When to Use It

- **Dynamic graph:** edges are added over time, need to know component count after each addition.
 - **Static graph with many connectivity queries:** DSU preprocesses, then each query is $O(\alpha(n))$.
 - **Number of Islands type problems:** activate cells, count connected groups.
-

4. Core Concepts

4.1 Tracking Component Count

```
class DSU {
    int components;
    // ...
    void unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return;
        // ... union logic
        components--;
    }
};
```

4.2 Checking if Graph is Connected

Graph is connected if `components == 1` (after all unions).

4.3 Checking if Graph is a Tree

Graph is a tree if:

- `components == 1` (connected)
 - Number of edges == $n - 1$
-

5. Step-by-Step

Problem: Start with $n=5$ isolated nodes. Add edges: (0,1), (2,3), (1,2), (3,4). Track components.

1. Initial: `components = 5`
2. Union(0,1): `components = 4`

3. Union(2,3): **components** = 3
4. Union(1,2): **components** = 2
5. Union(3,4): **components** = 1 (all connected)

6. Dry Run

Step	Edge	Root of u	Root of v	Same set?	Components
0	-	-	-	-	5
1	(0,1)	0	1	No	4
2	(2,3)	2	3	No	3
3	(1,2)	0	2	No	2
4	(3,4)	2	4	No	1

7. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class DSUComponents {
    vector<int> parent, sz;
    int comps;
public:
    DSUComponents(int n) : comps(n) {
        parent.resize(n);
        sz.resize(n, 1);
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }

    void unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return;
        if (sz[rx] < sz[ry]) swap(rx, ry);
        parent[ry] = rx;
        sz[rx] += sz[ry];
        comps--;
    }

    int components() { return comps; }
    bool isConnected() { return comps == 1; }
};
```

8. Complexity

- **Initialization:** $O(n)$
 - **Each union:** $O(\alpha(n))$
 - **Component count query:** $O(1)$
 - **Space:** $O(n)$
-

Cycle Detection (using DSU)

1. Overview

In an undirected graph, adding an edge between two vertices that are already in the same connected component creates a cycle. DSU can detect this in $O(\alpha(n))$ per edge.

2. Intuition

If two nodes already have a path connecting them, adding a direct edge between them creates a cycle — there are now two ways to go from one to the other.

3. When to Use It

- **Undirected graphs only** (DSU does not work for directed cycle detection).
 - Processing edges one by one and checking if graph remains acyclic.
 - Kruskal's MST (skip edges that create cycles).
 - Finding redundant connections.
-

4. Core Concepts

```
bool hasCycle = false;
for (auto &[u, v] : edges) {
    if (dsu.find(u) == dsu.find(v)) {
        hasCycle = true; // adding (u,v) creates a cycle
        break;
    }
    dsu.unite(u, v);
}
```

5. Step-by-Step

Graph: $n=4$, edges: (0,1), (1,2), (2,3), (0,3)

1. Edge (0,1): $\text{find}(0)=0, \text{find}(1)=1 \rightarrow \text{different} \rightarrow \text{unite} \rightarrow \text{sets: } \{0,1\}$

2. Edge (1,2): find(1)=0, find(2)=2 → different → unite → sets: {0,1,2}
3. Edge (2,3): find(2)=0, find(3)=3 → different → unite → sets: {0,1,2,3}
4. Edge (0,3): find(0)=0, find(3)=0 → same → **CYCLE DETECTED!**

6. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

bool hasCycle(int n, vector<pair<int,int>> &edges) {
    vector<int> parent(n), sz(n, 1);
    iota(parent.begin(), parent.end(), 0);

    function<int(int)> find = [&](int x) -> int {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    };

    auto unite = [&](int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return false;
        if (sz[rx] < sz[ry]) swap(rx, ry);
        parent[ry] = rx;
        sz[rx] += sz[ry];
        return true;
    };

    for (auto &[u, v] : edges) {
        if (!unite(u, v)) return true; // cycle found
    }
    return false;
}
```

7. Complexity

- $O((n + m) \times \alpha(n))$ where n = vertices, m = edges.
- Space: $O(n)$.

8. Common Mistakes

- Using DSU for directed cycle detection (doesn't work — use DFS with visited states).
- Not checking if the graph is already connected when adding edges.

DSU on Grid

1. Overview

DSU can be applied to 2D grids by mapping each cell (r, c) to a unique index $r * cols + c$. This allows tracking connectivity between cells, counting islands, merging regions, etc.

2. Intuition

Each cell is a node. Adjacent cells (up, down, left, right) that are both "active" are connected. By unioning active adjacent cells, DSU tracks connected components in the grid.

3. When to Use It

- **Number of Islands variants** where islands are added/removed dynamically.
 - **Grid connectivity problems** where cells become active over time.
 - **Problems involving merging regions** in a grid.
 - **Path existence** in a grid with obstacles.
-

4. Core Concepts

4.1 2D to 1D Mapping

```
int id(int r, int c, int cols) {  
    return r * cols + c;  
}
```

4.2 4-Directional Neighbors

```
int dr[] = {-1, 1, 0, 0};  
int dc[] = {0, 0, -1, 1};
```

4.3 Activating a Cell

```
void activate(int r, int c) {  
    int idx = r * cols + c;  
    active[idx] = true;  
    for (int d = 0; d < 4; d++) {  
        int nr = r + dr[d], nc = c + dc[d];  
        if (valid(nr, nc) && active[nr * cols + nc]) {  
            dsu.unite(idx, nr * cols + nc);  
        }  
    }  
}
```

5. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class DSUGrid {
    vector<int> parent, sz;
    vector<bool> active;
    int rows, cols, comps;

    int id(int r, int c) { return r * cols + c; }

public:
    DSUGrid(int r, int c) : rows(r), cols(c), comps(0) {
        parent.resize(r * c);
        sz.resize(r * c, 1);
        active.resize(r * c, false);
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }

    void unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return;
        if (sz[rx] < sz[ry]) swap(rx, ry);
        parent[ry] = rx;
        sz[rx] += sz[ry];
        comps--;
    }

    void activate(int r, int c) {
        int idx = id(r, c);
        if (active[idx]) return;
        active[idx] = true;
        comps++;

        int dr[] = {-1, 1, 0, 0};
        int dc[] = {0, 0, -1, 1};
        for (int d = 0; d < 4; d++) {
            int nr = r + dr[d], nc = c + dc[d];
            if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && active[id(nr,
nc)]) {
                unite(idx, id(nr, nc));
            }
        }
    }
}
```

```
int components() { return comps; }  
};
```

6. Dry Run

Grid: 3×3, activate cells: (1,1), (1,2), (2,1), (0,0)

Step	Cell	Active	Neighbors	Components
0	-	-	-	0
1	(1,1)	✓	none active	1
2	(1,2)	✓	(1,1) active → unite	1
3	(2,1)	✓	(1,1) active → unite	1
4	(0,0)	✓	none active	2

Final: 2 components — {(0,0)} and {(1,1), (1,2), (2,1)}

DSU with Rollback

1. Overview

DSU with rollback (also called reversible DSU or undoable DSU) supports undoing the most recent union operation. This is useful for offline dynamic connectivity problems where we process queries in a divide-and-conquer manner.

2. Intuition

Instead of permanently modifying parent and size arrays, we push the old values onto a stack before each change. When we need to rollback, we pop from the stack and restore the old values.

3. When to Use It

- **Offline dynamic connectivity:** edges are added and removed over time, process queries offline.
- **Divide and conquer on time intervals:** for problems where edges exist during certain time intervals.
- **Backtracking with DSU:** when you need to explore multiple branches of unions.

4. Core Concepts

4.1 Stack of Changes

Each change stores the index and old value of the modified location.

```
struct Change {
    int *ptr; // pointer to the modified variable
    int oldVal; // previous value
};
```

4.2 Union with Rollback

```
void unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx == ry) {
        history.push_back({nullptr, -1}); // marker for no-op
        return;
    }
    if (sz[rx] < sz[ry]) swap(rx, ry);
    // Save changes before modifying
    history.push_back({&parent[ry], parent[ry]});
    history.push_back({&sz[rx], sz[rx]});
    parent[ry] = rx;
    sz[rx] += sz[ry];
}
```

4.3 Rollback

```
void rollback() {
    while (!history.empty()) {
        Change c = history.back();
        history.pop_back();
        if (c.ptr == nullptr) break; // no-op marker
        *c.ptr = c.oldVal;
    }
}
```

5. Step-by-Step

1. Perform union operations as usual, pushing changes onto a stack.
2. Call `rollback()` to undo the most recent union.
3. Rollback restores `parent` and `size` arrays to their previous state.
4. Rollback can be called multiple times to undo multiple unions.

6. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```

struct DSUWithRollback {
    vector<int> parent, sz;
    vector<pair<int*, int>> history; // {pointer, oldValue}

    DSUWithRollback(int n) {
        parent.resize(n);
        sz.resize(n, 1);
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) {
        while (parent[x] != x) {
            // Can't do path compression with rollback (breaks history)
            x = parent[x];
        }
        return x;
    }

    void unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) {
            history.push_back({nullptr, -1}); // sentinel for no change
            return;
        }
        if (sz[rx] < sz[ry]) swap(rx, ry);
        // Save changes
        history.push_back(&parent[ry], parent[ry]);
        history.push_back(&sz[rx], sz[rx]);
        parent[ry] = rx;
        sz[rx] += sz[ry];
    }

    void rollback() {
        while (!history.empty()) {
            auto [ptr, oldVal] = history.back();
            history.pop_back();
            if (ptr == nullptr) break; // reached sentinel
            *ptr = oldVal;
        }
    }

    int getSize() { return history.size(); }
};

```

7. Complexity

Operation	Complexity
<code>find</code>	$O(\log n)$ — no path compression, but union by size keeps trees shallow

Operation	Complexity
<code>unite</code>	$O(\log n) + O(1)$ per change saved
<code>rollback</code>	$O(k)$ where k = number of changes undone
Space	$O(n) + O(\text{number of unions})$

8. Common Mistakes

- **Path compression breaks rollback** because it modifies nodes not involved in the current union. DSU with rollback typically skips path compression.
- Forgetting to save the old value **before** modifying.
- Not handling the no-op case properly (when `find(x) == find(y)`).
- Calling rollback more times than unions performed.

Persistent DSU

1. Overview

Persistent DSU allows querying the state of the DSU at any previous version. Each union operation creates a new version, and you can query `find` or `connected` on any historical version.

2. Intuition

Instead of modifying a single array, we store the entire history of each element's parent. Each version is a snapshot of the DSU at that point in time.

3. When to Use It

- Problems that ask "were nodes u and v connected at time t ?"
- Historical queries about connectivity.
- Very rare in placements, uncommon but known in CP.

4. Core Concepts

4.1 Versioned Arrays

Instead of `parent[i]`, we store a list of `(version, value)` pairs for each node.

```
vector<vector<pair<int,int>>> parent; // parent[i] = list of (version,
parentValue)
```

4.2 Finding at a Given Version

Find the value of `parent[i]` at the latest version $\leq$ the query version.

```
int getValue(int node, int version) {
    auto &vec = parent[node];
    auto it = upper_bound(vec.begin(), vec.end(), make_pair(version, INT_MAX));
    if (it == vec.begin()) return node; // default: self
    return prev(it)->second;
}
```

5. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class PersistentDSU {
    vector<vector<pair<int,int>>> parent; // parent[i] = list of (version, value)
    vector<vector<pair<int,int>>> sz; // sz[i] = list of (version, value)
    int currentVersion;
    int n;

    int getValueAtVersion(const vector<vector<pair<int,int>>> &arr, int idx, int
ver) {
        // Binary search for the latest version  $\leq$  ver
        auto &vec = arr[idx];
        auto it = upper_bound(vec.begin(), vec.end(), make_pair(ver, INT_MAX));
        if (it == vec.begin()) return -1; // should not happen for parent
        return prev(it)->second;
    }

    int findAtVersion(int x, int ver) {
        int p = getValueAtVersion(parent, x, ver);
        if (p == x) return x;
        // Path compression not possible in persistent DSU (would create new
version)
        return findAtVersion(p, ver);
    }

public:
    PersistentDSU(int n) : n(n), currentVersion(0) {
        parent.resize(n);
        sz.resize(n);
        for (int i = 0; i < n; i++) {
            parent[i].push_back({0, i});
            sz[i].push_back({0, 1});
        }
    }

    void unite(int x, int y) {
        currentVersion++;
    }
}
```

```

int rx = findAtVersion(x, currentVersion - 1);
int ry = findAtVersion(y, currentVersion - 1);
if (rx == ry) return;

int szx = getValueAtVersion(sz, rx, currentVersion - 1);
int szy = getValueAtVersion(sz, ry, currentVersion - 1);
if (szx < szy) swap(rx, ry);

// Create new version entries
parent[ry].push_back({currentVersion, rx});
sz[rx].push_back({currentVersion, szx + szy});
}

bool connected(int x, int y, int version) {
    return findAtVersion(x, version) == findAtVersion(y, version);
}

int getVersion() { return currentVersion; }
};

```

6. Complexity

Operation	Complexity
<code>unite</code>	$O(\log n \times \log V)$ — find is $O(\log n)$ per recursive call, times $\log V$ for binary search
<code>connected(x, y, ver)</code>	$O(\log n \times \log V)$
Space	$O(n + V)$ where V = number of versions
Notes	No path compression (would create too many versions)

7. Persistent DSU vs DSU with Rollback

Feature	Persistent DSU	DSU with Rollback
Query any version	✓ Yes	✗ No
Undo only most recent	✗	✓ Yes
Memory	$O(nV)$	$O(n + U)$
Speed	$O(\log n \log V)$	$O(\log n)$
Common in CP	Rare	Moderate

Final Notes

DSU is one of the most elegant and useful data structures. Master these key points:

1. **Always use path compression + union by size/rank** — it's just 2-3 extra lines and makes all operations nearly $O(1)$.
2. **DSU is for undirected graphs** — for directed graphs, use DFS or Tarjan's algorithm.
3. **DSU on grid** is a common pattern — map 2D to 1D.
4. **Reverse processing** is a powerful technique — when edges are removed, process backwards as additions.
5. **Rollback DSU** and **Persistent DSU** are advanced variations for specific problems.